Assignment 1 - Git & GitHub Fundamentals

Report

Full Name: Ali El Addouli
Student ID: 162617
Course Code & Section: CSC 3324 - Section 01
GitHub Username: Aliiiii5
Repository: https://github.com/Aliiiii5/git-session1-Aliiiii5

---

1. Repository Setup

I created the project **git-session1-Aliiiii5** from the starter files and ran `git init` to turn it into a Git repository. My `.gitignore` ignores `local.env` and all `*.log` files, with the exception `!important.log` so that file stays visible to Git. I checked it with `git check-ignore -v` and `git ls-files`: `local.env`, `app.log` and `debug.log` are ignored, while `important.log` is tracked.

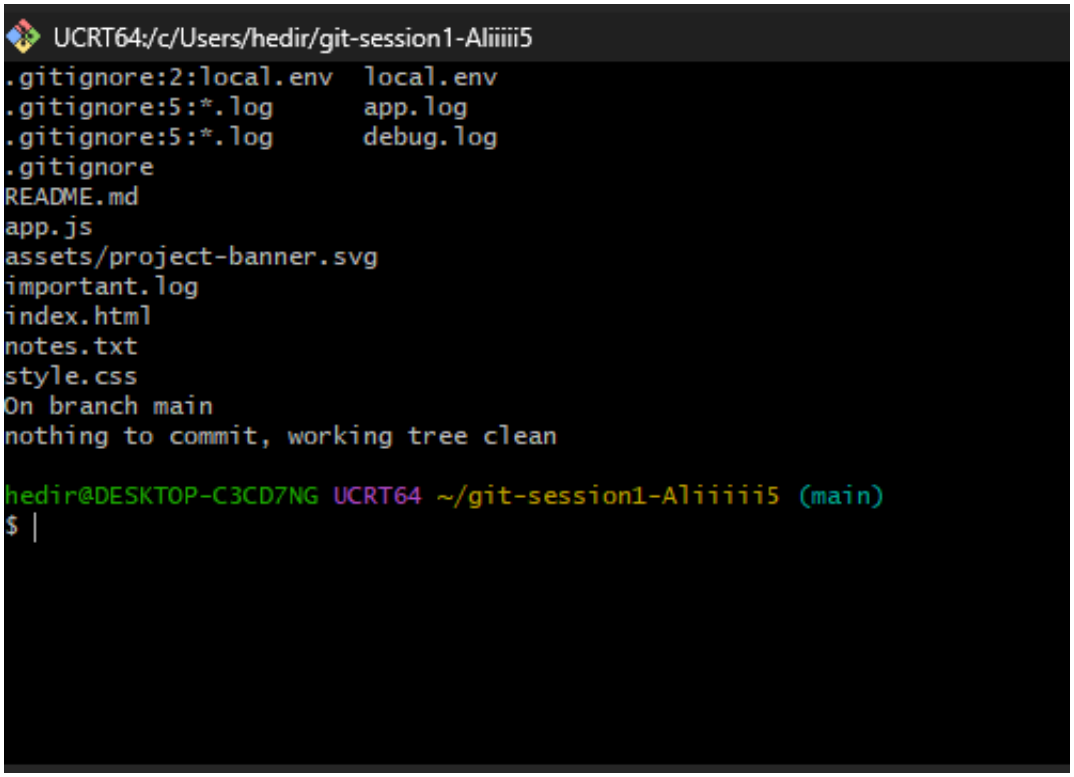


Figure 1 - Ignore state. Commands: `git check-ignore -v`, `git ls-files`, `git status`.

2. README & Commit History

My `README.md` contains my name and GitHub username, a one-sentence description, a technologies list, a code block (`git status`), a link to the Git documentation and the project banner image, and it renders correctly on GitHub. I made 10 commits in total (9 locally and 1 edited on GitHub), each one a single logical change with a short message: `.gitignore`, starter files, README, a header comment in `style.css`, a comment in `app.js`, the homepage, the notes, the About page, the project description fix, and a final README line. They touch at least 7 different files.

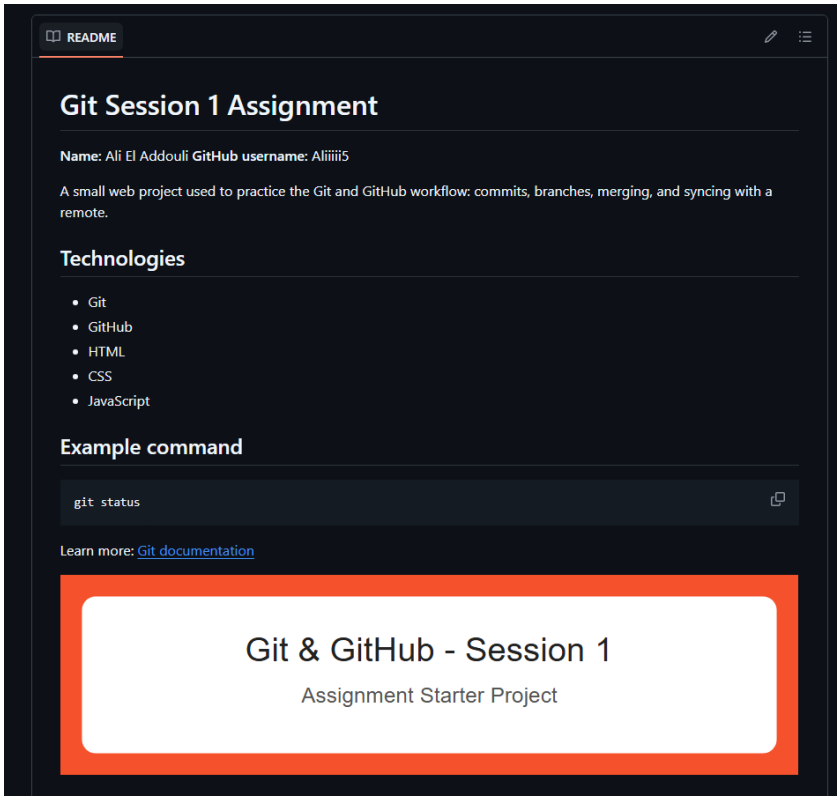


Figure 2 - Rendered README on GitHub.

```
UCRT64:/c/Users/hedir/git-session1-Aliiiii5
* 50a6585 (HEAD -> main) Add last updated line to README
* 4f76afc (origin/main, origin/HEAD) Update project description for clarity
* 2a25a3b (about-feature) Add About page
* feaa8a2 Update notes
* ef84125 Update homepage
* 3853dac Add comment to app.js
* 771f2eb Add header comment to style.css
* 9daf284 Add README with project description
* 9b9cbb6 Add starter project files
* 3e9cfba Add .gitignore for env and log files

hedir@DESKTOP-C3CD7NG UCRT64 ~/git-session1-Aliiiii5 (main)
$
```

Figure 3 - Commit history (`git log --oneline --graph --all --decorate`), taken just before the final push.

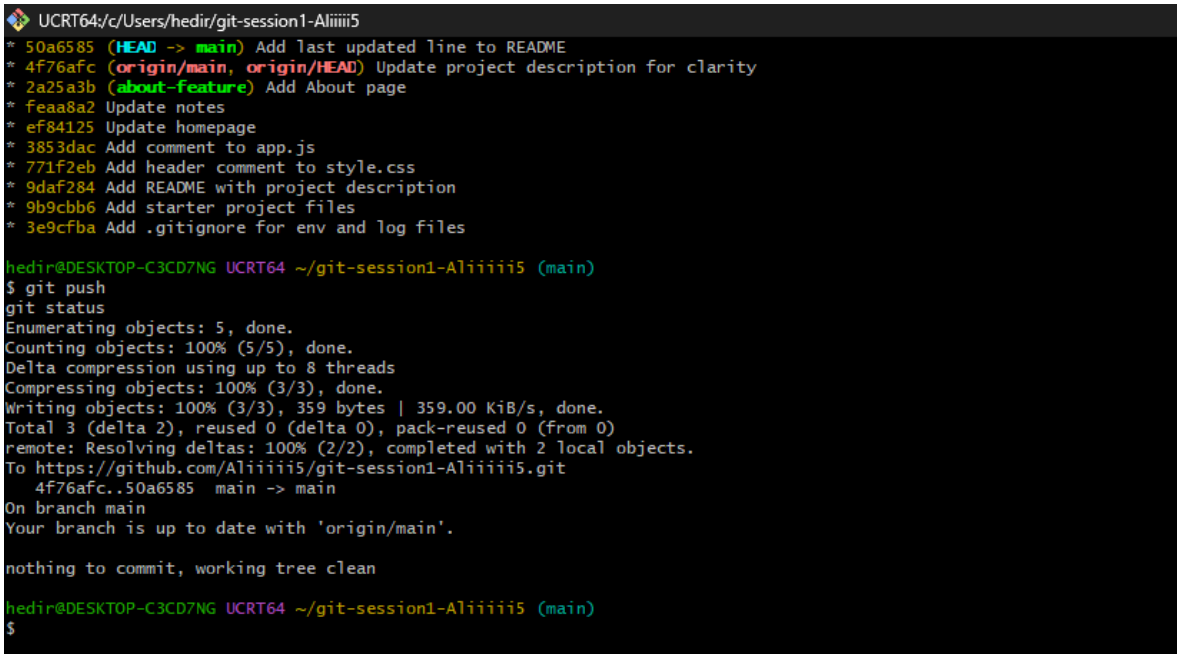


Figure 4 - Final push: `git push` followed by `git status` showing main is up to date with origin/main.

3. Selective Staging

I modified `index.html` and `notes.txt` at the same time, then staged only `index.html` with `git add index.html`. `git status` showed `index.html` under "Changes to be committed" and `notes.txt` under "Changes not staged for commit". I committed with `git commit -m "Update homepage"`; `git status` afterwards still showed `notes.txt` as modified, and I committed it afterwards as its own commit ("Update notes").

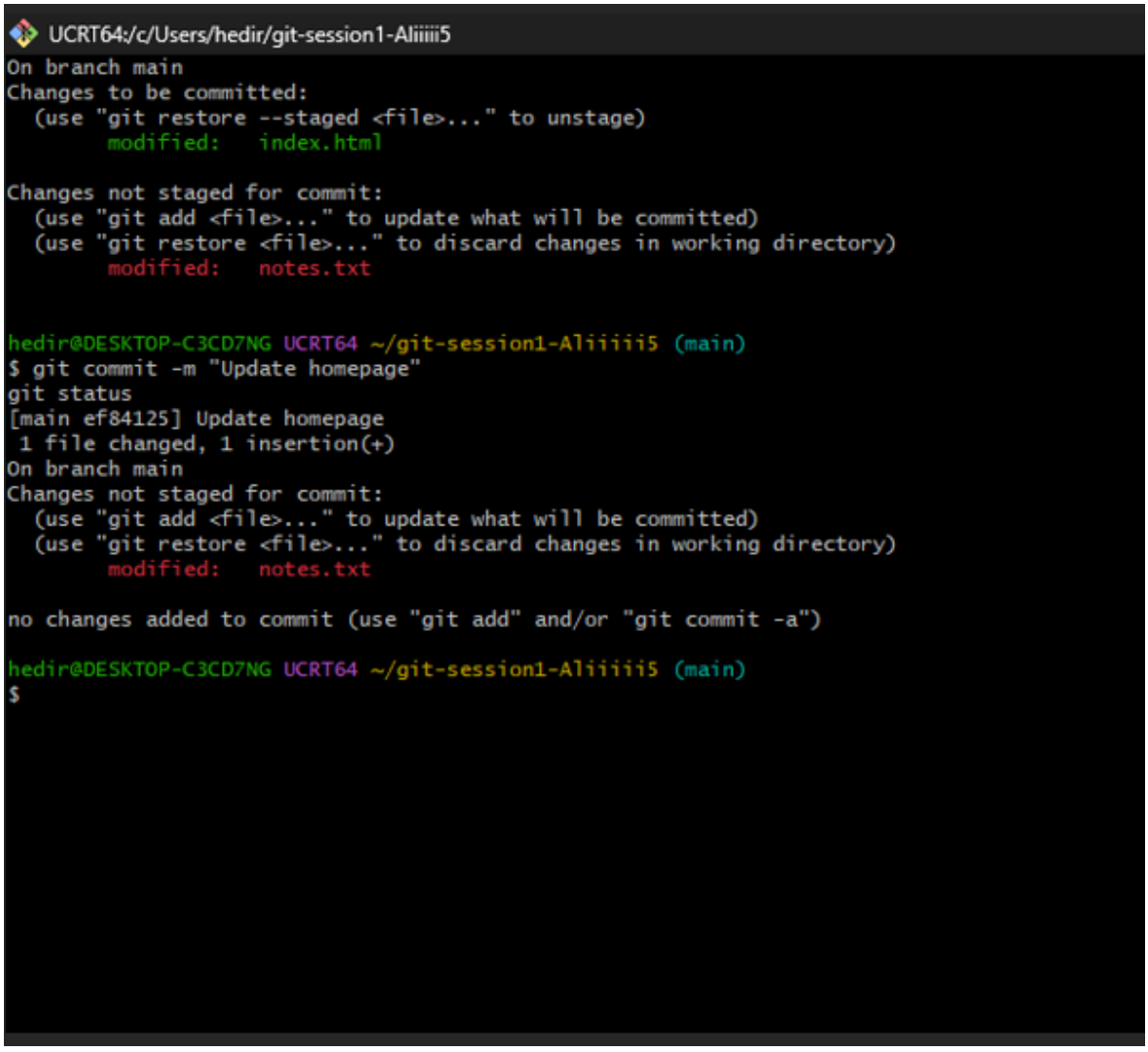


Figure 5 - One file staged and committed, the second file left modified.

4. Branch & Merge

I created the `about-feature` branch, committed "Add About page" on it, switched back to `main`, and ran `git merge about-feature`. Because `main` had no new commits since the branch was created, Git did a fast-forward merge: no merge commit was needed, and both `main` and `about-feature` now point to the same commit (`2a25a3b`).

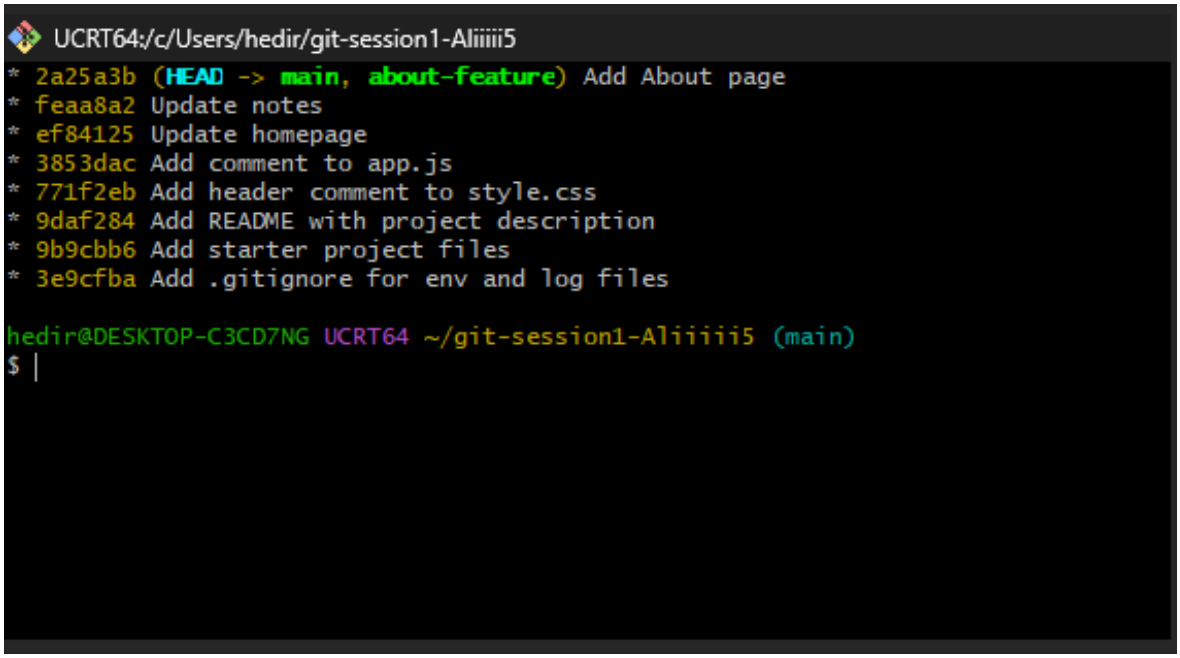


Figure 6 - Branch/merge history (`git log --oneline --graph --all --decorate`): HEAD -> main and about-feature on the same commit.

5. Remote Changes

A **commit** saves the staged changes as a snapshot in my local repository only, while a **push** uploads my local commits to the remote repository on GitHub (`git push -u origin main` published `main`). After I edited `README.md` directly on GitHub, my computer did not have that commit yet. `git fetch` downloads the remote information and updates the remote-tracking branch `origin/main` without touching my working files: `git status` then said my branch was behind `origin/main` by 1 commit, and the log showed the remote commit `4f76afc`. `git pull` does a fetch followed by a merge, so it brought that commit into my local `main`. Finally I added a line to `README.md`, inspected it with `git diff` before staging, staged it with `git add`, inspected it again with `git diff --staged`, then committed and pushed.

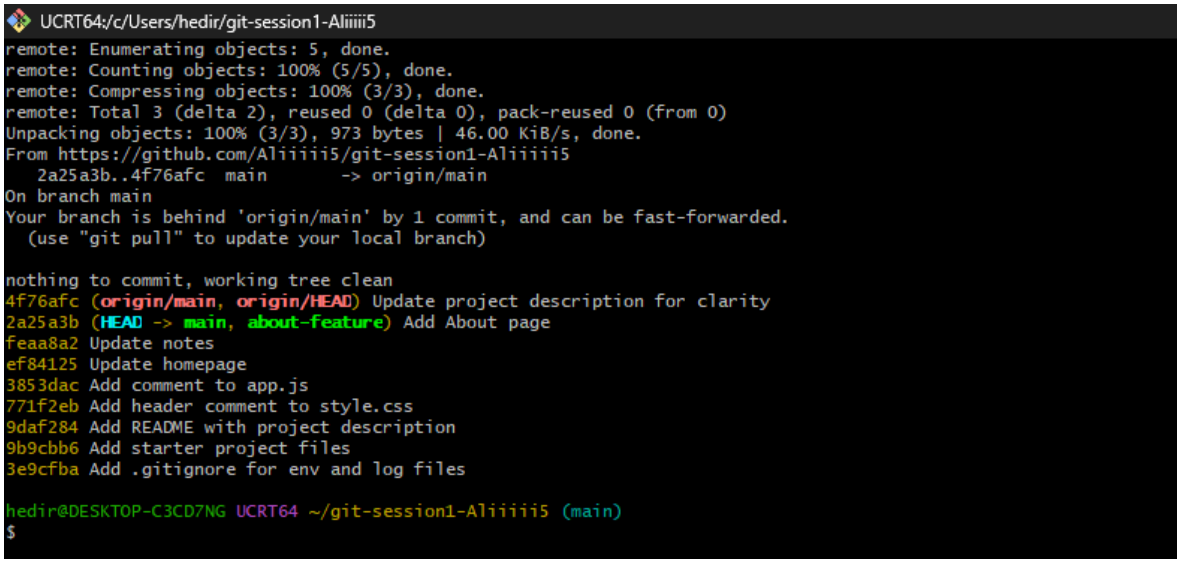


Figure 7 - After `git fetch`: local main is behind origin/main by 1 commit and the working tree is unchanged.

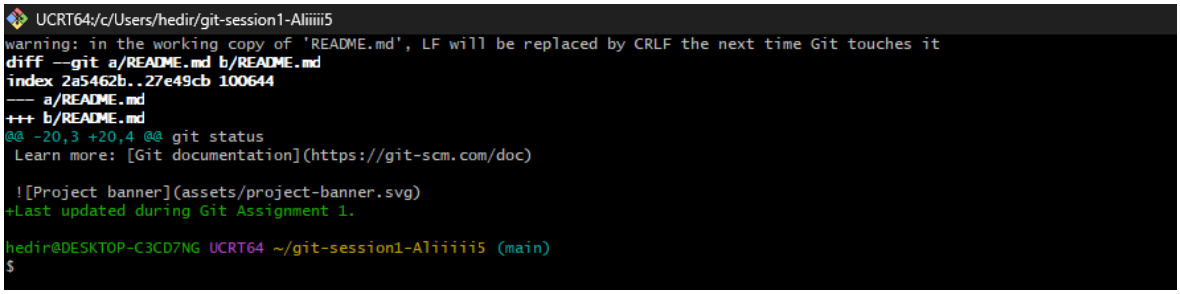


Figure 8 - Unstaged diff (`git diff`).

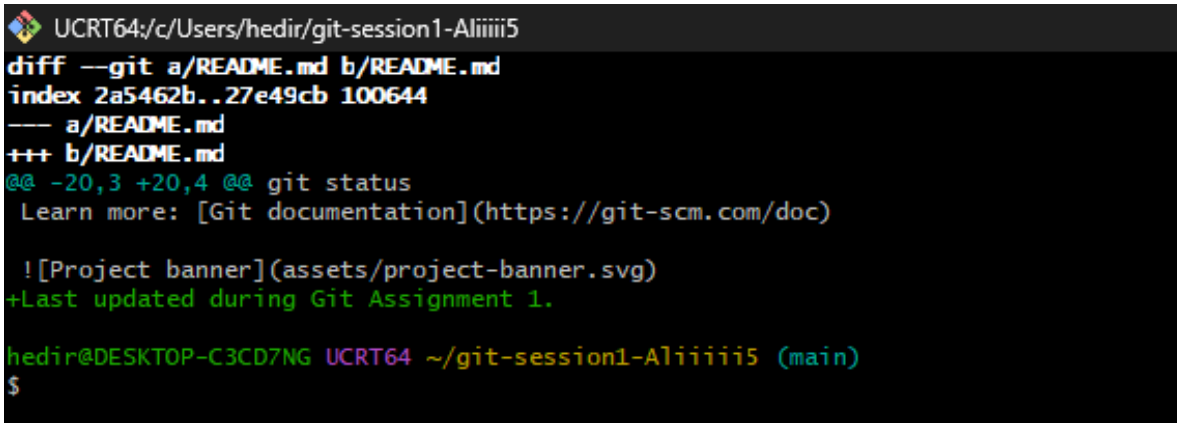


Figure 9 - Staged diff (`git diff --staged`).

6. Fair Use of AI

I used an AI assistant (Claude) as a checking and explanation tool; I ran all the Git and GitHub steps myself in Git Bash and on GitHub. I asked it which screenshots would prove each requirement of the assignment, how to verify my repository against the checklist (for example `git status -sb`, `git ls-files`, `git check-ignore -v`, `git log --graph` and `git branch --merged`), which requirement each of my screenshots proved and which ones were missing, and for help wording the report sections. I accepted the verification commands and the screenshot checklist. The review also showed that my last README commit had not been pushed yet, so I ran `git push` and confirmed with `git status` that my branch was up to date with `origin/main`. I reviewed the suggested wording against what I actually did and kept only what matched my work. I verified the answers by running every command myself, comparing the output with the Session 1 slides and the assignment instructions, and checking the result on GitHub. I can explain every command used in this submission.

7. Sources & Reflection

Sources: Session 1 course slides (Git Basics & Your First GitHub Repository); Assignment 1 instructions and report template; Claude (AI assistant), as described above.

Reflection: The hardest concept for me was the difference between `git fetch` and `git pull`, and how `origin/main` relates to my local `main`. `git fetch` only updates Git's remote-tracking reference, which is why my files stayed unchanged while `git status` said I was behind, whereas `git pull` also merges the remote commit. When Git behaves unexpectedly, I first run `git status` and `git log --oneline --graph --all --decorate` to check which branch I am on and how my working directory, staging area and remote compare, before trying any other command.